

Amazon RDS PostgreSQL Database Logging

To Begin with the Lab

Summary of the Lab

In this lab, PostgreSQL database logging was explored using Amazon RDS. SQL statements were executed from a PostgreSQL client session running on a Bastion Host, and the generated logs were viewed directly from the RDS console. The session-level logging parameter was modified to capture all SQL statements, allowing queries and errors to appear in the database logs. The lab demonstrated how PostgreSQL logging works, how logs can be viewed in real time from the RDS console, and how database administrators can use these logs for troubleshooting, auditing, and performance analysis.

Understanding PostgreSQL Database Logging in Amazon RDS

PostgreSQL logging records database activity such as SQL queries, user connections, warnings, and errors. These logs help database administrators understand what is happening inside the database and troubleshoot issues quickly. By controlling logging parameters, administrators can decide whether to log all statements, only errors, or specific events. This provides visibility into database operations without requiring additional monitoring tools.

Example:

A developer reports that an application query is failing. By enabling statement logging and reviewing the database logs, the administrator can identify the exact SQL query being executed and any associated error messages. This makes it easier to diagnose the problem and implement a fix.

Lab Steps

Step 1 – Create any Database with monitoring enabled

- I will create the Free Tier Database for lab purpose.
- Select **Aurora PostgreSQL** or **Amazon RDS PostgreSQL** based on the lab setup.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Availability zone group [info](#)

Availability zone group
Select the availability zone where you want to create your database.

US East (N. Virginia)

Engine options

Engine type [info](#)

☐ Aurora (MySQL Compatible)
 ☐ Aurora (PostgreSQL Compatible)
 ☐ MySQL
 ☒ PostgreSQL

☐ MariaDB
 ☐ Oracle
 ☐ Microsoft SQL Server
 ☐ IBM Db2

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.
 ☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

- Choose the current production version.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Templates

Choose a sample template to meet your use case.

☐ Production
Use defaults for high availability and fast, consistent performance.
 ☐ Dev/Test
This instance is intended for development use outside of a production environment.
 ☒ Free tier
Use the RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability

Deployment options [info](#)
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. [Learn more about the Amazon RDS service-level agreement \(SLA\).](#)

☐ Multi-AZ DB cluster deployment (3 instances)
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
 • 99.95% uptime
 • Redundancy across Availability Zones
 • Increased read capacity
 • Reduced write latency

☐ Multi-AZ DB instance deployment (2 instances)
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
 • 99.95% uptime
 • Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)
Creates a single DB instance without standby instances. This setup provides:
 • 99.5% uptime
 • No data redundancy

- Set the database identifier.
- Configure the master username and password.
- Attach the required security group.
- Set **Public Access** to **Yes**.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.
Default VPC (vpc-099792461c7d0a616)
11 Subnets, 7 Availability Zones
After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

After a database is created, you can't change its VPC.

DB subnet group [Info](#)
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.
default-vpc-099792461c7d0a616
6 Subnets, 6 Availability Zones

Public access [Info](#)
☒ Yes
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.
☐ No
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.
☒ Choose existing
Choose existing VPC security groups
☐ Create new
Create new VPC security group

Existing VPC security groups
Choose one or more options.
RDS-SG X default X

- Tick all three but we will be watching mainly **PostgreSQL log**.

With Performance Insights dashboard, you can visualize the database load on your Amazon RDS instance and filter the load by waits, SQL statements, hosts, or users.

Additional monitoring settings
Enhanced Monitoring, CloudWatch Logs and DevOps Guru

Enhanced Monitoring
☐ Enable Enhanced monitoring
Enabling Enhanced Monitoring metrics are useful when you want to see how different processes or threads use the CPU.

Log exports
Select the log types to publish to Amazon CloudWatch Logs.
☒ iam-db-auth-error log
☒ PostgreSQL log
☒ Upgrade log

IAM role
The following service-linked role is used for publishing logs to CloudWatch Logs.
RDS service-linked role

DevOps Guru
☐ Turn on DevOps Guru [Info](#)
DevOps Guru for RDS automatically detects performance anomalies for DB instances and provides recommendations.

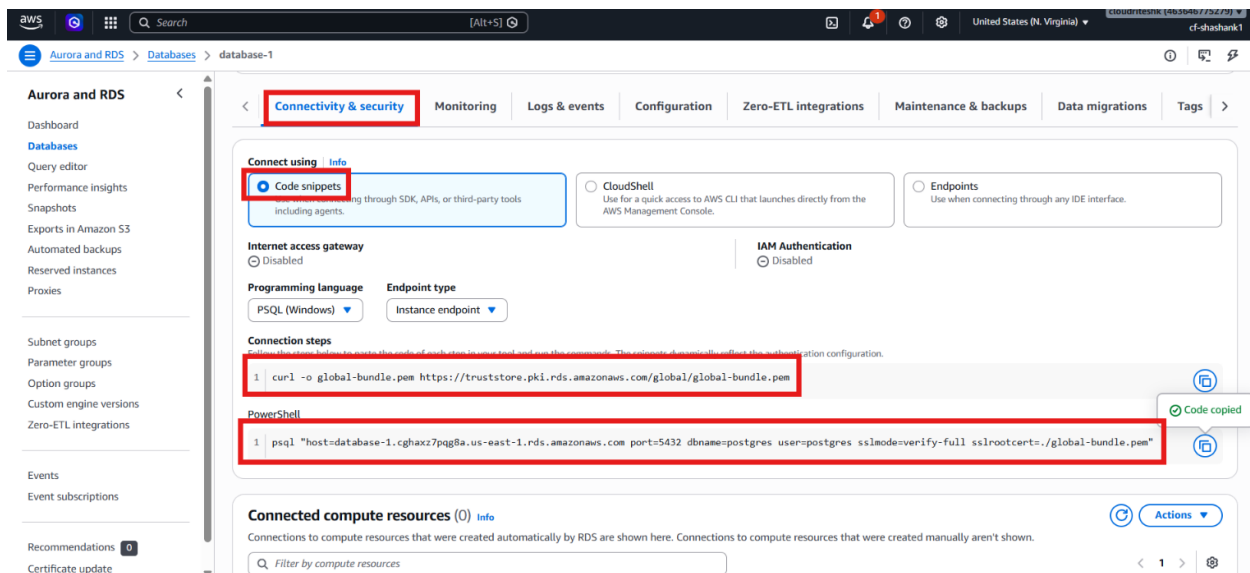
Additional configuration
Database options, encryption turned on, backup turned on, backtracking turned off, maintenance, CloudWatch Logs, delete protection turned off.

Estimated monthly costs
The Amazon RDS Free Tier is available to you for 12 months. Each calendar month, the free tier will allow you to use the Amazon RDS resources listed below for free:

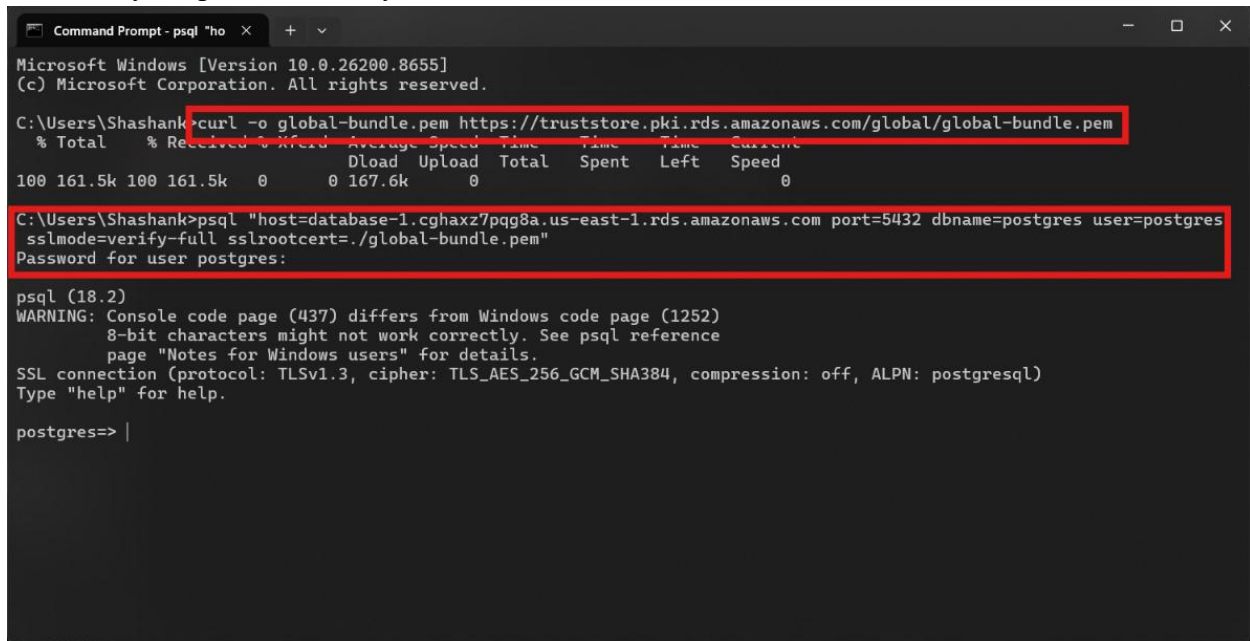
- Create the database and wait for it to become available.

Step 2 – Connect to the Database

- On the **Connectivity & security** panel connect using Code Snippets and run the commands in Command Prompt to connect to database.

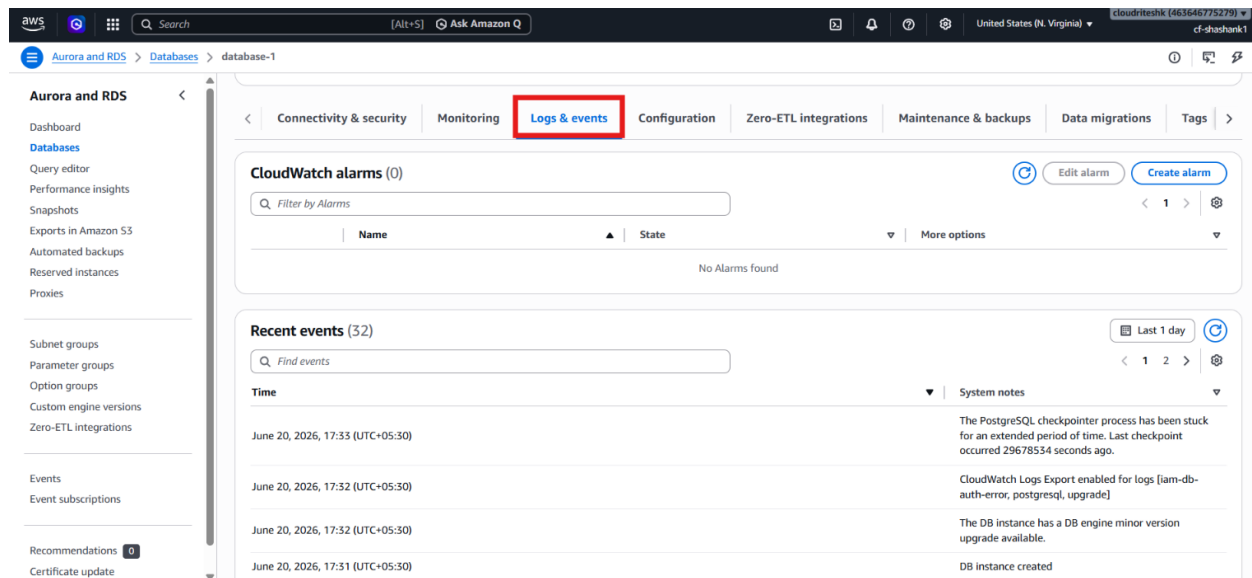


- Enter your password and you are connected to database.

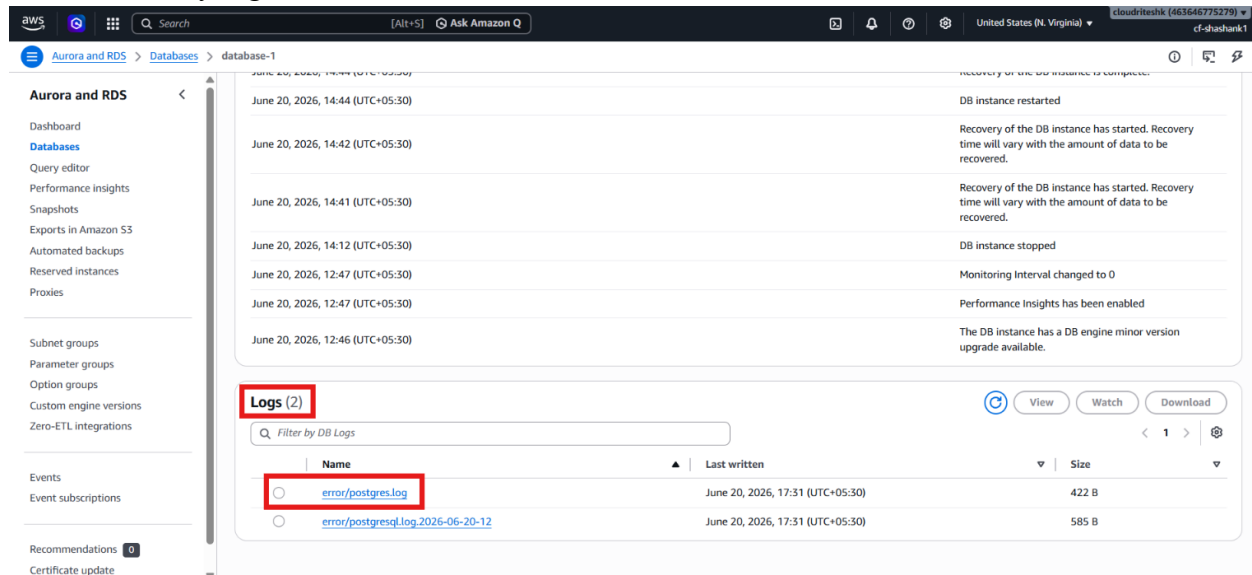


Step 3 – Watch Logs and Create tables

- Open your database and go to **Logs & Events** and scroll down to **Logs**.



- To view any log, click on it.



- After connecting to database, we will run some statements but none of them will be logged, so we must set log_statement to all to let it log all statements from now onwards.
- For this, we will run the following command and run statements and watch logs to see if anything changed on log.
- NOTE – The log statement will be valid for this session only, if you do disconnect it, you have to run this command again to log the statements in future.

```
SET log_statement='all';
```

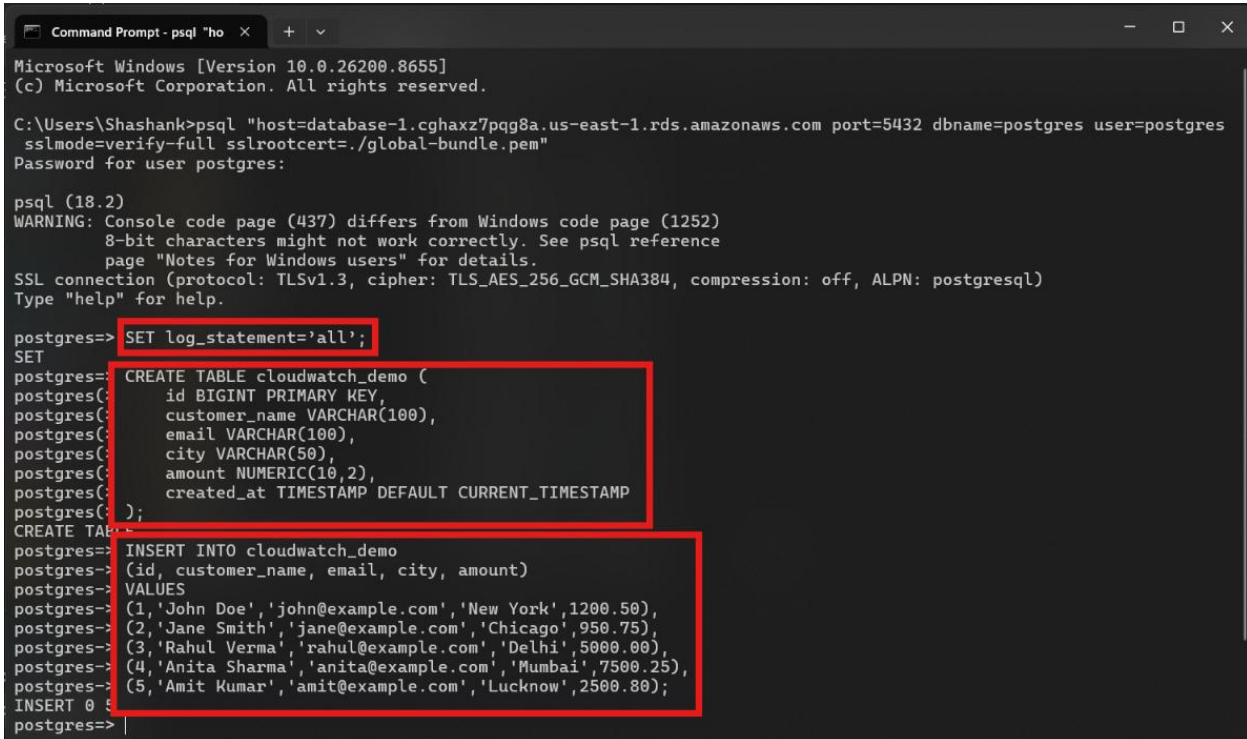
- Create tables and insert the values for lab purpose.

```
CREATE TABLE cloudwatch_demo (
  id BIGINT PRIMARY KEY,
  customer_name VARCHAR(100),
```

```
email VARCHAR(100),
city VARCHAR(50),
amount NUMERIC(10,2),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- Insert data

```
INSERT INTO cloudwatch_demo
(id, customer_name, email, city, amount)
VALUES
(1,'John Doe','john@example.com','New York',1200.50),
(2,'Jane Smith','jane@example.com','Chicago',950.75),
(3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
(4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
(5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
```



```
Command Prompt - psql "ho" X + v
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shashank>psql "host=database-1.cghaxz7pqg8a.us-east-1.rds.amazonaws.com port=5432 dbname=postgres user=postgres
sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user postgres:

psql (18.2)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> SET log_statement='all';
SET
postgres=> CREATE TABLE cloudwatch_demo (
postgres=>     id BIGINT PRIMARY KEY,
postgres=>     customer_name VARCHAR(100),
postgres=>     email VARCHAR(100),
postgres=>     city VARCHAR(50),
postgres=>     amount NUMERIC(10,2),
postgres=>     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
postgres=> );
CREATE TABLE
postgres=> INSERT INTO cloudwatch_demo
postgres=> (id, customer_name, email, city, amount)
postgres=> VALUES
postgres=> (1,'John Doe','john@example.com','New York',1200.50),
postgres=> (2,'Jane Smith','jane@example.com','Chicago',950.75),
postgres=> (3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
postgres=> (4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
postgres=> (5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
INSERT 0 5
postgres=>
```

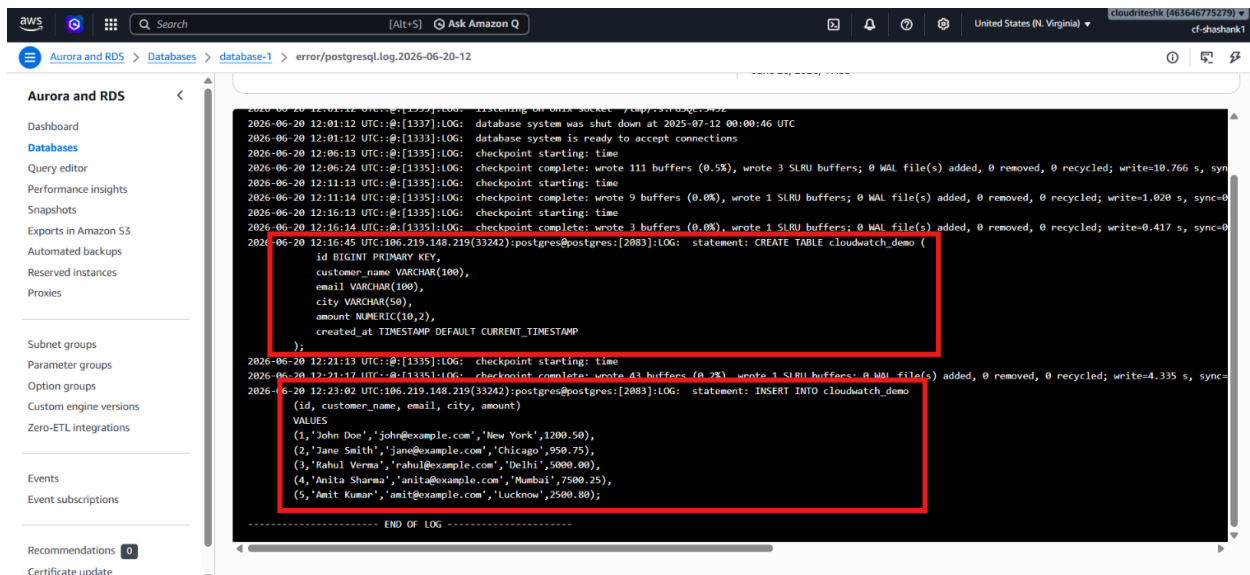
- Now go to logs, select it and click on Watch.

The screenshot shows the AWS Aurora and RDS console. On the left is a navigation menu with options like Dashboard, Databases, Query editor, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations, and Certificate update. The main content area shows the 'Logs (2)' section for 'database-1'. It includes a search bar 'Filter by DB Logs' and three buttons: 'View', 'Watch' (highlighted with a red box), and 'Download'. Below these is a table with columns 'Name', 'Last written', and 'Size'. The table contains two entries: 'error/postgres.log' (422 B) and 'error/postgresql.log.2026-06-20-12' (2.9 kB). The second entry is selected, and its name is also highlighted with a red box.

- Now we can see the log being updated automatically and we can also download it from here.

The screenshot shows the 'Logs details' page for the log file 'error/postgresql.log.2026-06-20-12'. The breadcrumb navigation shows 'Aurora and RDS > Databases > database-1 > error/postgresql.log.2026-06-20-12'. The log file name is 'error/postgresql.log.2026-06-20-12'. The 'Download' button is highlighted with a red box. The log content is displayed in a dark-themed text area, showing database system status and a CREATE TABLE statement. The log content includes timestamps and log messages such as 'database system was shut down at 2025-07-12 00:00:46 UTC', 'database system is ready to accept connections', 'checkpoint starting: time', 'checkpoint complete: wrote 111 buffers (0.5%), wrote 3 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=10.766 s, sync=0.000 s; checkpoint time=0.000 s', 'checkpoint starting: time', 'checkpoint complete: wrote 9 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=1.020 s, sync=0.000 s; checkpoint time=0.000 s', 'checkpoint starting: time', 'checkpoint complete: wrote 3 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.417 s, sync=0.000 s; checkpoint time=0.000 s', 'statement: CREATE TABLE cloudwatch_demo (id BIGINT PRIMARY KEY, customer_name VARCHAR(100), email VARCHAR(100), city VARCHAR(50), amount NUMERIC(10,2), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);', 'checkpoint starting: time', and 'checkpoint complete: wrote 43 buffers (0.2%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=4.335 s, sync=0.000 s; checkpoint time=0.000 s'.

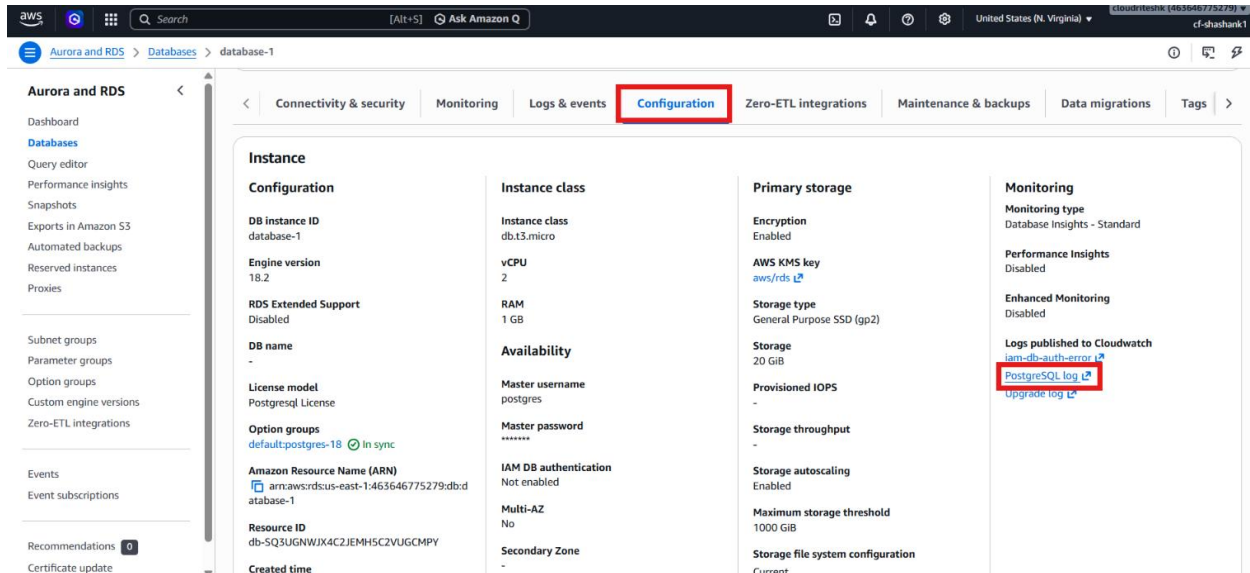
- Here we can see our statements getting logged as we ran them earlier.



- Here

Step 4 – Watch Logs in CloudWatch

- Earlier while creating the database, we checked the tickbox **PostgreSQL logs** for exporting logs to CloudWatch, so our logs are already being exported to CloudWatch.
- Now we will go to CloudWatch and see if our logs appeared there.
- Open the Configuration tab and click on **PostgreSQL log**.



- Now CloudFront will open the page and we can see the streams of our logs being generated.

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Timestamp	Message	Log stream name
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12.531 GMT [1333] LOG: skipping missing configuration file ...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12.531 GMT [1333] LOG: skipping missing configuration file ...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: redirecting log output to logging col...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:HINT: Future log output will appear in dir...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: starting PostgreSQL 18.2 on x86_64-pc...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: listening on IPv4 address "0.0.0.0", ...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: listening on IPv6 address ":::", port ...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: listening on Unix socket "/tmp/.s.PGSQL...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1337]:LOG: database system was shut down at 2025...	database-1.0
2026-06-20T12:01:12.000Z	2026-06-20 12:01:12 UTC:[1333]:LOG: database system is ready to accept co...	database-1.0
2026-06-20T12:06:13.000Z	2026-06-20 12:06:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:06:24.000Z	2026-06-20 12:06:24 UTC:[1335]:LOG: checkpoint complete: wrote 111 buffer...	database-1.0
2026-06-20T12:11:13.000Z	2026-06-20 12:11:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:11:14.000Z	2026-06-20 12:11:14 UTC:[1335]:LOG: checkpoint complete: wrote 9 buffers ...	database-1.0
2026-06-20T12:16:13.000Z	2026-06-20 12:16:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:16:14.000Z	2026-06-20 12:16:14 UTC:[1335]:LOG: checkpoint complete: wrote 3 buffers ...	database-1.0

- Scroll down and select one and we can see the query we have ran earlier with prefix **postgres@postgres::LOG:**

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Timestamp	Message	Log stream name
2026-06-20T12:16:14.000Z	2026-06-20 12:16:14 UTC:[1335]:LOG: checkpoint complete: wrote 3 buffers ...	database-1.0
2026-06-20T12:16:45.000Z	2026-06-20 12:16:45 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: stat...	database-1.0
2026-06-20 12:16:45 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: CREATE TABLE cloudwatch_demo (		
	id BIGINT PRIMARY KEY,	
	customer_name VARCHAR(100),	
	email VARCHAR(100),	
	city VARCHAR(50),	
	amount NUMERIC(10,2),	
	created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP	
	);	
2026-06-20T12:21:13.000Z	2026-06-20 12:21:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:21:17.000Z	2026-06-20 12:21:17 UTC:[1335]:LOG: checkpoint complete: wrote 43 buffers...	database-1.0
2026-06-20T12:23:02.000Z	2026-06-20 12:23:02 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG:...	database-1.0
2026-06-20T12:26:13.000Z	2026-06-20 12:26:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:26:15.000Z	2026-06-20 12:26:15 UTC:[1335]:LOG: checkpoint complete: wrote 9 buffers ...	database-1.0
2026-06-20T12:31:13.000Z	2026-06-20 12:31:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:31:13.000Z	2026-06-20 12:31:13 UTC:[1335]:LOG: checkpoint complete: wrote 2 buffers ...	database-1.0
2026-06-20T12:36:13.000Z	2026-06-20 12:36:13 UTC:[1335]:LOG: checkpoint starting: time	database-1.0
2026-06-20T12:36:13.000Z	2026-06-20 12:36:13 UTC:[1335]:LOG: checkpoint complete: wrote 2 buffers ...	database-1.0

- Here, we can see our CREATE TABLE query we ran earlier.
- To watch it log in real-time, click on Log management on left panel and click on your log group.

CloudWatch > Log management

Log groups (5)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Log group	Log class	Anomaly d...	Deletion prot...	Bearer token ...	Data protec...	Sensitive d...	Retention
/aws/lambda/create-ec2	Standard	Configure	Off	Off	-	-	Never exp
/aws/rds/instance/database-1/postgresql	Standard	Configure	Off	Off	-	-	Never exp
/aws/rds/proxy/proxy-postgresql	Standard	Configure	Off	Off	-	-	Never exp
/aws/sagemaker/studio	Standard	Configure	Off	Off	-	-	Never exp
RDSOSMetrics	Standard	Configure	Off	Off	-	-	1 month

- On the **Log Stream** tab, click on your log stream.

CloudWatch > Log management > /aws/rds/instance/database-1/postgresql

Log group details

Log class: Standard

ARN: [arn:aws:logs:us-east-1:463646775279:log-group:/aws/rds/instance/database-1/postgresql:*](#)

Creation time: -

Retention: Never expire

Stored bytes: -

Account: -

Metric filters: -

Subscription filters: Not supported

Contributor Insights rules: Not supported

KMS key ID: -

Deletion protection: Off

Data protection: -

Sensitive data count: Not supported

Custom field indexes: Not supported

Transformer: Not supported

Anomaly detection: Not supported

Bearer token authentication: Off

Syslog ingestion: [Info](#)

Log streams (1)

By default, we only load the most recent log streams.

Filter log streams or try prefix search

Log stream	Last event time
database-1.0	2026-06-20 12:31:13 (UTC)

- Now click on **Resume** so it will be capturing the logs again.

- Now we can run any statement, and it will get logged.
- We will run this query at the terminal 2 times.

```
SELECT * FROM cloudwatch_demo;
```

```

Command Prompt - psql "ho" x + v
postgres(> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
postgres(> );
CREATE TABLE
postgres=> INSERT INTO cloudwatch_demo
postgres=> (id, customer_name, email, city, amount)
postgres=> VALUES
postgres=> (1,'John Doe','john@example.com','New York',1200.50),
postgres=> (2,'Jane Smith','jane@example.com','Chicago',950.75),
postgres=> (3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
postgres=> (4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
postgres=> (5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
INSERT 0 5
postgres=> SELECT * FROM cloudwatch_demo;
 id | customer_name | email | city | amount | created_at
-----+-----+-----+-----+-----+-----
 1 | John Doe | john@example.com | New York | 1200.50 | 2026-06-20 12:23:02.251274
 2 | Jane Smith | jane@example.com | Chicago | 950.75 | 2026-06-20 12:23:02.251274
 3 | Rahul Verma | rahul@example.com | Delhi | 5000.00 | 2026-06-20 12:23:02.251274
 4 | Anita Sharma | anita@example.com | Mumbai | 7500.25 | 2026-06-20 12:23:02.251274
 5 | Amit Kumar | amit@example.com | Lucknow | 2500.80 | 2026-06-20 12:23:02.251274
(5 rows)

postgres=> SELECT * FROM cloudwatch_demo;
 id | customer_name | email | city | amount | created_at
-----+-----+-----+-----+-----+-----
 1 | John Doe | john@example.com | New York | 1200.50 | 2026-06-20 12:23:02.251274
 2 | Jane Smith | jane@example.com | Chicago | 950.75 | 2026-06-20 12:23:02.251274
 3 | Rahul Verma | rahul@example.com | Delhi | 5000.00 | 2026-06-20 12:23:02.251274
 4 | Anita Sharma | anita@example.com | Mumbai | 7500.25 | 2026-06-20 12:23:02.251274
 5 | Amit Kumar | amit@example.com | Lucknow | 2500.80 | 2026-06-20 12:23:02.251274
(5 rows)

postgres=> |

```

- And we can see our logs now.

aws

Search

[Alt+S]

Ask Amazon Q

United States (N. Virginia)

cloudriteshk [463646775279]

cf-shashank

CloudWatch

Log management

/aws/rds/instance/database-1/postgresql

database-1.0

CloudWatch

<

Favorites and recents

Ingestion

Dashboards

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

▼ Logs

Log Management

Log Analytics

Log Anomalies

▼ Metrics

All metrics

Query Studio

Explorer

Streams

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

Timestamp	Message
2026-06-20T12:36:13.000Z	2026-06-20 12:36:13 UTC:::[1335]:LOG: checkpoint starting: time
2026-06-20T12:36:13.000Z	2026-06-20 12:36:13 UTC:::[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.
2026-06-20T12:41:13.000Z	2026-06-20 12:41:13 UTC:::[1335]:LOG: checkpoint starting: time
2026-06-20T12:41:14.000Z	2026-06-20 12:41:14 UTC:::[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.
2026-06-20T12:46:13.000Z	2026-06-20 12:46:13 UTC:::[1335]:LOG: checkpoint starting: time
2026-06-20T12:46:13.000Z	2026-06-20 12:46:13 UTC:::[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.
2026-06-20T12:51:13.000Z	2026-06-20 12:51:13 UTC:::[1335]:LOG: checkpoint starting: time
2026-06-20T12:51:14.000Z	2026-06-20 12:51:14 UTC:::[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.
2026-06-20 12:55:37 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;	
2026-06-20 12:55:37 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;	
2026-06-20T12:56:13.000Z	2026-06-20 12:56:13 UTC:::[1335]:LOG: checkpoint starting: time
2026-06-20T12:56:13.000Z	2026-06-20 12:56:13 UTC:::[1335]:LOG: checkpoint complete: wrote 3 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0.
2026-06-20T12:56:20.000Z	2026-06-20 12:56:20 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;
2026-06-20 12:56:20 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;	

No newer events at this moment. Auto retrying... Pause

Back to top